Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 3

The Command Line: Shell, Pipes and Text Filters

Track A: The Real OS, from the Outside (Linux)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. Move Fast, Find Help **2**

2.1 Observe answers 2

3 Part 2. What the Shell Does to the Command Line **2**

3.1 Observe answers 3

4 Part 3. Redirection, Chaining and Jobs **3**

5 Part 4. Answering Questions with One-Liners **4**

5.1 Observe answers 4

6 Part 5. First Shell Script **5**

6.1 Observe answers 5

7 Conclusion **5**

1. Purpose of the Work

The purpose of this laboratory work is to develop practical Bash command-line skills and understand command lookup, shell expansion, quoting, redirection, chaining, jobs, pipelines, text filters, and a first reusable shell script. A synthetic `access.log` with 500 records was used for the text-processing tasks.

2. Part 1. Move Fast, Find Help

```
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
nosuchprog: command not found
exit code: 127
```

Figure 1.1 – Actual command-type and missing-command output

2.1. Observe answers

1. `cd` and `echo` are shell built-ins. `ls` is an alias that ultimately invokes `/usr/bin/ls`. `cd` must be a built-in because only the current shell can change its own working directory.
2. The inspected PATH contained 49 directories, and the external `ls` executable was in `/usr/bin`.
3. A missing command returned exit status 127.
4. The `ls` option for sorting by modification time is `-t`; newer files are listed first by default.

3. Part 2. What the Shell Does to the Command Line

```
a.c b.c
file1.txt file2.txt
a.c alfa beta.txt b.c
*.xyz
```

```
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
'alfa beta.txt'
$HOME /home/sofia $HOME
1 2 3 4 5
42
today is Monday, I am sofia
log1.txt  log2.txt  log3.txt
```

Figure 2.1 – Actual results of globbing, quoting and expansion experiments

3.1. Observe answers

1. Bash transforms many command lines before the selected program starts.
2. `echo` does not search the filesystem for `*.c`; Bash performs pathname expansion and passes the resulting names to `echo`.
3. `ls alfa beta.txt` produces two arguments. Valid fixes are `ls "alfa beta.txt"` and `ls alfa\ beta.txt`.
4. Single quotes preserve text literally, double quotes preserve spaces while allowing expansions, and a backslash escapes an individual special character.

4. Part 3. Redirection, Chaining and Jobs

```
$ ls /etc /nope > out.txt 2> err.txt
$ wc -l out.txt
184 out.txt
$ cat err.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.1 – Actual stdout and stderr redirection result

```
[1] 2384
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
```

Figure 3.2 – Actual background-job and termination output

The operators have different control-flow meanings: `;` always continues, `&&` continues only after success, and `||` continues only after failure. The order of `> all.txt 2>&1` and `2>&1 > all.txt` matters because descriptor duplication uses the destination that exists at the moment the duplication is performed.

5. Part 4. Answering Questions with One-Liners

```
100
/page0
/page1
/page2
/page3
Count: 4
125 /page3
125 /page2
125 /page1
125 /page0
15 user5
15 user3
Total: 72
Failed: 15
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.1 – Actual main results produced by oneliners.sh

5.1. Observe answers

1. There were 100 failed requests.
2. Four pages were requested: `/page0` through `/page3`.
3. Each page received 125 requests, giving a total of 500.
4. `user3` and `user5` had the maximum number of failed requests, 15 each.
5. `user3` made 72 requests, of which 15 failed.
6. The last three failed requests were `10:10 user0`, `10:15 user5`, and `10:20 user3`.

7. The inspected login-shell counts were 26 `/usr/sbin/nologin`, 2 `/bin/bash`, 1 `/bin/sync`, and 1 `/bin/false`.
8. `ls -l /etc | wc -l` was one line larger than `ls /etc | wc -l` because long-format `ls` adds a `total` summary line.

6. Part 5. First Shell Script

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.1 – Actual report.sh output for access.log

```
no such file: nope.log
exit code: 1
```

Figure 5.2 – Actual report.sh error handling for a missing file

6.1. Observe answers

1. `#!/bin/bash` is the shebang that selects Bash when the script is executed directly.
2. `./report.sh` explicitly names the current directory because `.` is normally not included in PATH.
3. `>&2` sends diagnostics to standard error, and `exit 1` makes failure detectable by the caller.

7. Conclusion

The laboratory showed that Bash performs command discovery, expansion, quoting, substitution and redirection before a target program runs. Pipelines

and small text-processing utilities can be composed to analyze structured data, and the final script combined these mechanisms with variables, conditionals, loops, stderr handling and exit statuses.